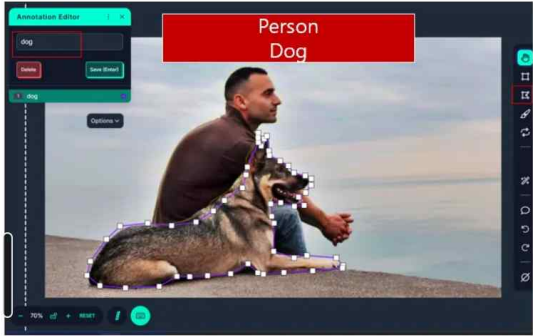


로보플로우 어노테이션 및 mask(y)값 제작법

로보플로우에서 COCO Segmentation을 제작 -> json 불러오기



Download

Image and Annotation Format

COCO Segmentation

JSON annotations used with Detectron 2.

Download Options

로보플로우에서 COCO Segmentation으로 저장

```
1
2 import json
3
4 json_path = "/content/_annotations.coco.json"
5
6 with open(json_path, "r") as f:
7     coco = json.load(f)
8     coco.keys()

dict_keys(['info', 'licenses', 'categories', 'images', 'annotations'])

1 coco['categories']

[{'id': 0, 'name': 'person', 'supercategory': 'none'},
 {'id': 1, 'name': 'dog', 'supercategory': 'person'},
 {'id': 2, 'name': 'person', 'supercategory': 'person'}]

1 len(coco['annotations'])

2
```

key	용도
images	이미지 크기(height, width)
annotations	segmentation 값(= 마스크를 만드는 핵심)
categories	라벨 번호(category_id → 클래스 이름)

- image_id → 어떤 이미지의 마스크인지 찾기
- category_id → 이 polygon이 어떤 클래스인지
- segmentation → 폴리곤 점들 → 실제 mask 영역
- 'images' key는 mask의 크기를 만들어주는 정보

```
{
  "id": 3,
  "file_name": "xxxxx.jpg",
  "width": 640,
  "height": 480
}
```

```

1 import json
2 import numpy as np
3 import cv2
4
5 json_path = "/content/_annotations.coco.json"
6
7 with open(json_path, "r") as f:
8     coco = json.load(f)
9
10 images = coco["images"]
11 annotations = coco["annotations"]
12
13 # 넘파이 출력 옵션 세팅 (생략 없음)
14 np.set_printoptions(threshold=np.inf)
15
16
17 # 이미지 1개 선택
18 img_info = images[0]
19 h, w = img_info["height"], img_info["width"]
20 img_id = img_info["id"]
21
22 # 해당 이미지의 annotation 찾기
23 anns = [a for a in annotations if a["image_id"] == img_id]
24
25 # 마스크 초기화
26 mask = np.zeros((h, w), dtype=np.uint8)

```

$$\begin{array}{r} (98,) \\ (49, 2) \\ \hline (106,) \\ (53, 2) \\ \hline (312, 312) \end{array}$$
$$\begin{pmatrix} 106, \\ 53, 2 \end{pmatrix}$$
 $(312, 312)$ [illegible]

2

```
3 import matplotlib.pyplot as plt
```

```
4 import numpy as np
```

5

```
6 colored = np.zeros((mask.shape[0], mask.shape[1], 3), dtype=np.uint8)
```

7

```
8 colored[mask == 1] = [255, 0, 0] # 빨강
```

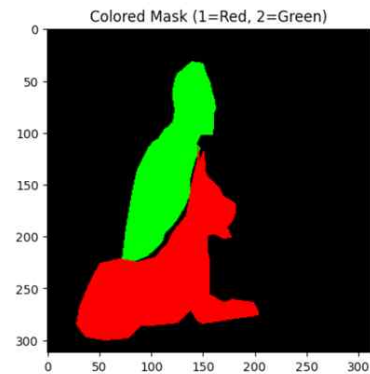
```
9 colored[mask == 2] = [0, 255, 0] # 초록
```

10

```
11 plt.imshow(colored)
```

```
12 plt.title("Colored Mask (1=Red, 2=Green)")
```

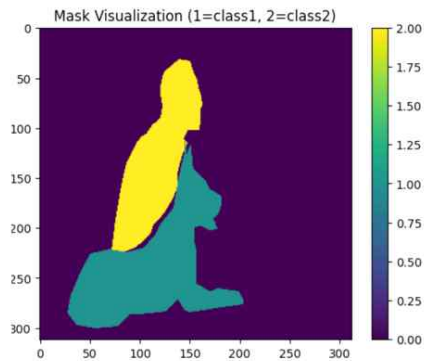
```
13 plt.show()
```



```

1 import matplotlib.pyplot as plt
2
3 plt.imshow(mask)
4 plt.colorbar()
5 plt.title("Mask Visualization (1=class1, 2=class2)")
6 plt.show()

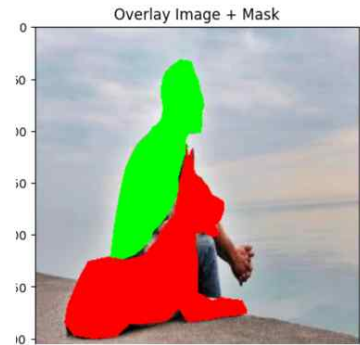
```



```

1 # -----
2 # 원본 오버레이
3 # -----
4 import cv2
5 import matplotlib.pyplot as plt
6
7 img = cv2.imread("/content/test_seg.jpg.rf_e6759a95aa5048f65d21be27b0685d5c.jpg")
8 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
9
10 overlay = img.copy()
11 overlay[mask == 1] = (255, 0, 0)
12 overlay[mask == 2] = (0, 255, 0)
13
14 plt.imshow(overlay)
15 plt.title("Overlay Image + Mask")
16 plt.show()

```



● SAM(Segment Anything Model)이란

“점(Point), 박스(Box), 텍스트 등 간단한 프롬프트만으로 마스크를 자동 생성해주는 모델.”

컴퓨터가 아닌 사람이 어디를 segment 하고 싶은지 “힌트”만 주면 즉시 마스크 만들어줌.

- 파이썬 코드로도 구현가능 (‘파이썬 이미지분할 SAM’으로 검색)
- 웹에서의 SAM (구글 검색 ‘웹 SAM’, ‘허깅페이스 Web SAM’)

(1) Segment Anything Playground (Meta AI 공식)

— “Upload image → 클릭/점 찍기 → 마스크 생성” 가능

(2) Segment Anything Web (Hugging Face 스페이스) — 브라우저에서 바로 실행 가능.

(3) SAM 2 Demo (Meta 이후 버전 영상/이미지용) — 영상/동작 포함된 고급 버전.

확률변수 (Random Variable) 와 확률분포 (Probability Distribution)

세그멘테이션이 하는 일: 이미지의 각 픽셀이 "강아지일까? 고양이일까? 배경일까?"를 판단하는 것

확률변수로 보면: 각 픽셀마다 "이게 뭘 확률이 얼마나 될까?"를 계산하는 것

→ 세그멘테이션 신경망 = 각 픽셀이 무엇일지 확률을 계산

신경망에서는

입력: 사진 한 장

↓

[신경망이 분석]

↓

출력: 각 픽셀마다

- 강아지일 확률: 80%
- 고양이일 확률: 15%
- 배경일 확률: 5%

왜 확률로 표현할까?

1. 확신의 정도를 알 수 있음

- "이 픽셀은 90% 확률로 강아지" → 확신함
- "이 픽셀은 51% 확률로 강아지" → 애매함

2. 모든 가능성을 고려함

- 딱 잘라 "강아지다!"보다
- "강아지 같긴 한데 고양이일 수도..." 같은 애매한 상황도 표현 가능

1. 확률의 기본

- 확률은 0~1 사이의 값
- 모든 경우의 확률을 더하면 1
- 예: $P(\text{강아지}) + P(\text{고양이}) + P(\text{배경}) = 1$

4. 최대우도 (Maximum Likelihood)

"가장 확률이 높은 것을 선택"

- 위 예시에서 픽셀 A는 "사람"으로 분류

2. 조건부 확률

$P(Y|X) = "X가 주어졌을 때 Y일 확률"$

세그멘테이션에서는:

- $P(\text{이 픽셀이 강아지} | \text{이미지})$
- "이 이미지를 봤을 때, 이 픽셀이 강아지일 확률"

5. 크로스 엔트로피 (Cross Entropy)

"예측한 확률분포가 정답과 얼마나 다른가"

- 신경망 학습할 때 사용
- 값이 작을수록 예측을 잘한 것

↓
손실값

3. 확률분포

여러 결과에 대한 확률들의 집합

픽셀 A의 확률분포:

- 배경: 10%
- 사람: 70%
- 자동차: 20%

6. 불확실성 (Uncertainty)

모델이 얼마나 확신하는가

- 확률이 90%면 → 확신함
- 확률이 55%면 → 불확실함

1 확률이란

"어떤 일이 일어날 가능성"을 숫자로 표현한 것입니다.

예시로 이해하기:

- 동전을 던졌을 때 앞면이 나올 확률 = 0.5 (50%)
- 주사위에서 6이 나올 확률 = $1/6 \approx 0.167$ (16.7%)
- 규칙 1: 확률은 0과 1 사이
 - 불가능한 일: 0 / 확실한 일: 1 / 애매한 일: 0과 1 사이
- 규칙 2: 모든 경우를 더하면 1
 - 주사위 예시: $P(1) + P(2) + P(3) + P(4) + P(5) + P(6) = 1$: $1/6 + 1/6 + 1/6 + 1/6 + 1/6 + 1/6 = 1$
 - 날씨 예시: $P(\text{맑음}) + P(\text{흐림}) + P(\text{비}) = 1$: $0.5 + 0.3 + 0.2 = 1$
- 규칙 3: 확률은 퍼센트로도 표현 가능

세그멘테이션에서는?

이미지의 한 픽셀이 무엇인지 판단할 때:

이 픽셀이...

- 하늘일 확률: 0.7 (70%)
 - 건물일 확률: 0.2 (20%)
 - 나무일 확률: 0.1 (10%)
- 합계: 1.0 (100%) ✓

1-1. 확률변수 X: 확률공간의 사건을 실수(real number)에 대응시키는 함수 $X: \Omega \rightarrow \mathbb{R}$

사건이 발생하면 그 사건에 대응하는 숫자가 확률변수의 값

- Ω (오메가, sample space, 표본공간)는 확률 실험에서 가능한 모든 결과들의 집합
확률을 논리적으로 다루기 위해서는 "무슨 사건이 가능한지"를 먼저 정의
 - $\Omega = \{H, T\} \rightarrow$ 동전던지기에서 앞(Heads), 뒤(Tails)
가능한 모든 경우: HH, HT, TH, TT $\rightarrow \Omega = \{HH, HT, TH, TT\}$
 - $\Omega = \{1, 2, 3, 4, 5, 6\} \rightarrow$ 주사위던지기에서 숫자 1-6
- $X(\omega)$: 실제 사건 ω 가 일어났을 때 확률변수 값:

왜 Ω (오메가)인가? 확률론을 만든 콜모고로프의 표기 관습

현대 확률론을 공리화한 사람은 안드레이 콜모고로프(1933)임, 그가 표본공간을 Ω 로 표기
 Ω 는 "전체 / 우주(universe)"를 상징하는 문자: 그리스 문자 Ω (Omega)는 알파벳의 마지막 글자
그래서 종종 "전체" / "끝까지 포함된 집합" / "우주(전체 세계)" 같은 의미로 사용됨
표본공간은 가능한 모든 결과를 한데 모은 "전체 공간"임으로 $\rightarrow \Omega$ 가 상징적으로 잘 맞는 기호

- 확률변수는 그 값이 취할 수 있는 형태에 따라 두 가지로 나뉩니다.

종류	정의	예시
이산 확률변수 (Discrete RV)	값이 셀 수 있는 유한개 또는 가산 무한개로 구성됨 (정수처럼 띄엄띄엄한 값)	주사위 눈, 합격자 수, 하루 불량품 개수
연속 확률변수 (Continuous RV)	값이 연속 구간 전체의 실수가 될 수 있음 (길이, 무게처럼 연속 값)	사람의 키, 온도, 센서 측정 오차

1-2. 조건부 확률 (Conditional Probability)

"어떤 조건이 주어졌을 때의 확률"입니다.

표기법: $P(A|B)$ = "B가 일어났을 때 A가 일어날 확률"

- $|$ 이 기호는 "~가 주어졌을 때" 또는 "~라는 조건에서"를 의미합니다

예시 1: 날씨와 우산

$P(\text{우산 들고 다님} | \text{비 올}) = 0.9$ (90%) → "비가 올 때, 우산을 들고 다닐 확률"

$P(\text{우산 들고 다님} | \text{맑음}) = 0.1$ (10%) → "맑을 때, 우산을 들고 다닐 확률"

예시 2: 카드 뽑기

52장 카드에서: $P(\text{하트}) = 13/52 = 0.25$ → 그냥 하트 뽑을 확률

$P(\text{하트} | \text{빨간색 카드}) = 13/26 = 0.5$ → "빨간색이라는 조건"이 주어지면 하트 확률이 올라감!

세그멘테이션에서의 조건부 확률

핵심: $P(\text{레이블} | \text{이미지})$

픽셀(100, 200)위치에서: $P(\text{하늘} | \text{이미지의 윗부분, 파란색}) = 0.9$ → 윗부분이고 파란색이면 하늘 확률 높음

$P(\text{하늘} | \text{이미지의 아랫부분, 회색}) = 0.1$ → 아랫부분이고 회색이면 하늘일 확률 낮음

1-3. 기대값 (Expectation 또는 Expected Value) : 기호 $E[X]$

- **확률변수**: 픽셀 하나가 어떤 클래스일지 → 결과가 랜덤하게 정해지는 변수
- **조건부 확률**: 이 픽셀 값(RGB)을 봤을 때, 고양이일 확률 $P(\text{고양이}|\text{픽셀})$
- **기대값**: 확률을 기반으로 예측한 평균적인 결과

| 각 픽셀마다 고양이일 확률이 51%, 이런 픽셀이 1000개 있다고 하면...

- 이 중 약 51%, 즉 $0.51 \times 1000 = 510$ 개 정도는 실제 고양이 / 나머지 490개는 고양이가 아님

즉, 확률=0.51이면 실제로 맞는 비율도 약 51%로 수렴한다는 의미.

기댓값(E)은 $E=1000 \times 0.51=510$, (기값: 바로 확률 $\times$ 총 개수)

- 확률이 0.51이라는 건 "1을 낼 기대 비율이 0.51인 베르누이 확률변수

따라서 1000개의 독립 픽셀은

$$X_i \sim \text{Bernoulli}(0.51)$$

- 픽셀 하나를 1(고양이), 0(배경)으로 본다면 이 픽셀은 베르누이 분포($\text{Bernoulli}(p)$)를 따름.

이때 합은은

$$S = \sum X_i, \quad E[S] = 1000 \times 0.51$$

- 베르누이 분포: 성공(1) 또는 실패(0)이라는 두 가지 결과만 있는 한 번의 실험을 모델링하는 가장 기본적인 확률 분포

- softmax output 또는 sigmoid output에서 얻는 확률값은 각 픽셀의 **기대를 의미하는 값**이기 때문에:

모두 "이 확률은 기댓값"임

- 픽셀 단위 예측 확률을 해석할 때
- 평균 IoU, Dice 계산할 때
- 보정(calibration) 할 때
- threshold 0.5 기준 잡을 때

- $P=0.51$ 인 픽셀 1000개
→ 기대적으로 510개가 정답

- $P=0.9$ 인 픽셀 1000개
→ 기대적으로 900개가 정답

- 이는 확률 $\times$ 개수 = **베르누이 확률변수의 기댓값**

- 세그멘테이션의 확률 맵은 모두 이런 "기댓값" 해석이 가능함

1-4. 최대 우도 (Likelihood)

- **우도**: 모델이 정답을 맞힐 확률들의 곱
- **최대우도추정**: 우도를 최대화하는 파라미터 찾기
- **Cross-Entropy**: 음의 로그 우도 → 최소화 대상
 - Cross-Entropy를 최소화하는 것 = 최대우도추정입니다."

픽셀	모델 A (P=고양이)	모델 B (P=고양이)	최대우도 선택
1	0.9	0.6	모델 A
2	0.8	0.7	모델 A
3	0.5	0.65	모델 B

좋은 모델 → 정답을 높은 확률로 예측하는 모델

"수학적으로 표현하면, **정답이 나올 확률을 최대화**하는 모델입니다."

각 픽셀에서 정답이 나올 확률을 다 곱하면?

우도(Likelihood) 계산

- 모델 A: $0.9 \times 0.8 \times 0.85 = 0.612$
- 모델 B: $0.6 \times 0.7 \times 0.65 = 0.273$

- **최대우도(Maximum Likelihood Estimation, MLE)**는 주어진 데이터에 대해 가장 높은 확률을 가지는 모수(parameter)를 추정하는 통계적 방법 : 우도를 최대화하는 파라미터를 찾는 것
→ "즉, 학습이란 **정답들이 나올 확률을 최대한 높이는 방향**으로 모델을 조정하는 겁니다."

- 문제점: "픽셀이 수만 개면 확률을 계속 곱해야 하는데..."

$0.9 \times 0.8 \times 0.85 \times 0.7 \times \dots \times 0.9$ "숫자가 너무 작아짐, 컴퓨터가 계산을 못"

. 로그를 취하면 "곱셈을 덧셈으로 바꿔서 계산함" $\log(0.9 \times 0.8 \times 0.85) = \log 0.9 + \log 0.8 + \log 0.85$

"우도를 최대화 → 로그 우도를 최대화 → 음의 로그 우도를 최소화" $-\log P = \text{Cross-Entropy Loss}$

1-5. 크로스 엔트로피

모델	P(고양이)	P(배경)	예측 클래스
A	0.9	0.1	고양이
B	0.6	0.4	고양이

손실 함수(Loss Function)의 역할

두 모델이 있습니다.

둘 다 이 픽셀을 고양이로 분류함. '얼마나 잘했는지'를 숫자로 표현

→ 수학적으로 표현 Cross-Entropy

정답이 고양이(클래스 1)일 때 손실

$$\text{Loss} = -\log P(\text{고양이})$$

P(고양이)	손실 $-\log P$
0.9	0.105
0.6	0.511
0.1	2.303

"손실은 예측이 정답에서 얼마나 벗어났는지를 측정

- 정답이 고양이(1)인데 0.9로 예측 → 조금 틀림
- 정답이 고양이(1)인데 0.6으로 예측 → 더 많이 틀림
- 정답이 고양이(1)인데 0.1로 예측 → 심하게 틀림

확률이 낮을수록, 즉 확신 없이 맞혔거나 틀렸을수록 손실이 급격히 커집니다